

Руководство по развертыванию проекта F1-F2

Содержание

1. Подготовка окружения
2. Установка PostgreSQL
3. Клонирование проекта
4. Настройка виртуального окружения
5. Настройка базы данных
6. Запуск сервера
7. Сброс и обновление данных
8. Часто задаваемые вопросы

Подготовка окружения

Необходимое ПО

- Python 3.9+ (скачать с python.org)
- Git (скачать с git-scm.com)
- PostgreSQL 16+ (скачать с postgresql.org)

Проверка установки

После установки откройте командную строку (cmd) или PowerShell и проверьте:

```
bash
```

- `python --version`
- *# Должно быть: Python 3.9.x или выше*
-
- `git --version`
- *# Должно быть: git version 2.x.x*
-
- `psql --version`

```
# Должно быть: psql (PostgreSQL) 13.x или выше
```

Установка PostgreSQL

Для Windows:

1. Скачайте установщик с [официального сайта](#)
2. Запустите установку
3. Важно! При установке укажите пароль: 12345678
4. Оставьте порт по умолчанию: 5432
5. Завершите установку

Проверка установки PostgreSQL

bash

- *# Войдите в PostgreSQL*
- `psql -U postgres`
-
- *# Должны увидеть приглашение: postgres=#*

Выйти: \q

Клонирование проекта

Вариант 1: Если проект на GitHub/GitLab

bash

- *# Перейдите в папку, где будет проект*
- `cd C:\Users\%USERNAME%\PycharmProjects\`
-
- *# Склонируйте репозиторий*
- `git clone https://github.com/ваш-репозиторий/kartotekaProject.git`
-
- *# Перейдите в папку проекта*

`cd kartotekaProject`

Вариант 2: Копирование с флешки/диска

bash

- *# Просто скопируйте папку проекта в нужное место*

Например: C:\Users\%USERNAME%\PycharmProjects\kartotekaProject

Настройка виртуального окружения

Создание виртуального окружения

bash

- *# Перейдите в папку проекта*
- `cd C:\Users\%USERNAME%\PycharmProjects\kartotekaProject`
-
- *# Создайте виртуальное окружение*
- `python -m venv venv`
-
- *# Активируйте виртуальное окружение (Windows)*
- `venv\Scripts\activate`
-

Должны увидеть (venv) в начале строки

Установка зависимостей

bash

- *# Убедитесь, что вы в виртуальном окружении (должно быть (venv))*
-
- *# Обновите pip*
- `python -m pip install --upgrade pip`
-
- *# Установите зависимости*
- `pip install django`
- `pip install psycopg2-binary`
- `pip install chardet`

```
pip install networkx
```

Проверка установки

```
bash
```

- pip list
- # Должны увидеть:
- # Django
- # psycopg2-binary
- # chardet

```
# networkx
```

Настройка базы данных

Создание базы данных

```
bash
```

- # Войдите в PostgreSQL
- psql -U postgres
-
- # Введите пароль: 12345678
-
- # Создайте базу данных (выполните в psql)
- CREATE DATABASE kartoteka_db;
-
- # Проверьте, что база создалась
- \l
-
- # Выйдите из psql

```
\q
```

Применение миграций

bash

- *# Убедитесь, что вы в папке проекта и виртуальное окружение активно*
- `cd C:\Users\%USERNAME%\PycharmProjects\kartotekaProject`
-
- *# Примените миграции*
- `python manage.py migrate`
-
- *# Должны увидеть:*
- *# Applying core.0001_initial... OK*

и т.д.

Создание суперпользователя (admin)

bash

- *# Создайте администратора*
- `python manage.py createsuperuser`
-
- *# Введите:*
- *# Username: admin*
- *# Email: (оставьте пустым, нажмим Enter)*
- *# Password: 12345678*
- *# Password (again): 12345678*
-

Должны увидеть: Superuser created successfully.

Загрузка начальных данных (если есть)

bash

- *# Если есть файл с данными (fixtures)*
- `python manage.py loaddata initial_data.json`
-

Или через импорт в интерфейсе (позже)

Запуск сервера

Запуск в режиме разработки

bash

- *# Убедитесь, что вы в папке проекта и виртуальное окружение активно*
- `cd C:\Users\%USERNAME%\PycharmProjects\kartotekaProject`
-
- *# Запустите сервер*
- `python manage.py runserver`
-
- *# Должны увидеть:*

Starting development server at http://127.0.0.1:8000/

Проверка работы

Откройте браузер и перейдите по адресам:

1. Главная страница: <http://127.0.0.1:8000/>
2. Админка: <http://127.0.0.1:8000/admin/>
 - Логин: admin
 - Пароль: 12345678
3. Руководство: <http://127.0.0.1:8000/user-guide/>

Сброс и обновление данных

Важно! Эти команды НЕ УДАЛЯЮТ ваши данные, а только обновляют структуру

Обновление после изменений в коде

```
bash
```

- *# Остановите сервер (Ctrl+C)*
-
- *# Примените новые миграции (данные сохраняются!)*
- `python manage.py migrate`
-
- *# Соберите статические файлы (для изображений)*
- `python manage.py collectstatic`
-
- *# Запустите сервер снова*

```
python manage.py runserver
```

Если нужно обновить структуру БД (данные сохраняются)

```
bash
```

- *# 1. Создайте новую миграцию (если были изменения в моделях)*
- `python manage.py makemigrations`
-
- *# 2. Примените миграции (данные НЕ теряются!)*

```
python manage.py migrate
```

Если нужно перезагрузить статические файлы

bash

- *# После добавления новых скриншотов или CSS*
- `python manage.py collectstatic`
-

Ответьте 'yes' если спросит про перезапись

Полный сброс кэша (данные не теряются)

bash

- *# Просто перезапустите сервер - кэш очистится сам*

`python manage.py runserver`

Полезные команды для терминала

Работа с виртуальным окружением

bash

- *# Активировать окружение (Windows)*
- `venv\Scripts\activate`
-
- *# Активировать окружение (Mac/Linux)*
- `source venv/bin/activate`
-
- *# Деактивировать окружение*
- `deactivate`
-
- *# Узнать, активен ли venv (должен быть (venv) в начале строки)*

Просто посмотрите на приглашение в терминале

Работа с PostgreSQL

bash

- *# Войти в PostgreSQL*
- `psql -U postgres`
-
- *# Показать все базы данных*
- `\l`
-
- *# Подключиться к базе проекта*
- `\c kartoteka_db`
-
- *# Показать все таблицы*
- `\dt`
-
- *# Показать структуру таблицы*
- `\d core_object`
-
- *# Выйти*

`\q`

Работа с Git (если используется)

bash

- *# Забрать последние изменения (данные сохраняются!)*
- `git pull`
-
- *# Посмотреть статус*
- `git status`
-
- *# Откатить изменения, если что-то сломалось*

`git checkout -- .`

Работа с Django

bash

- *# Создать резервную копию данных*
- `python manage.py dumpdata > backup.json`
-
- *# Восстановить данные из резервной копии*
- `python manage.py loaddata backup.json`
-
- *# Очистить кэш*
- `python manage.py clear_cache`
-
- *# Запустить тесты (проверка работоспособности)*

`python manage.py test`

Часто задаваемые вопросы

Что делать, если порт 8000 уже занят?

```
bash
```

- *# Запустите на другом порту*
- `python manage.py runserver 8080`
- *# или*

```
python manage.py runserver 127.0.0.1:8001
```

Как сбросить пароль администратора?

```
bash
```

- *# Войдите в оболочку Django*
- `python manage.py shell`
-
- *# Выполните команды:*
- `from django.contrib.auth.models import User`
- `user = User.objects.get(username='admin')`
- `user.set_password('12345678')`
- `user.save()`

```
exit()
```

Как проверить, работает ли база данных?

```
bash
```

- *# Войдите в PostgreSQL*
- `psql -U postgres`
-
- *# Проверьте подключение*
- `\conninfo`
-
- *# Проверьте наличие базы*

```
\l
```

Потеряются ли данные при обновлении?

Нет! Данные сохраняются при:

- `git pull` — только код, не БД
- `python manage.py migrate` — только структура
- `python manage.py collectstatic` — только файлы
- Перезапуске сервера

Данные теряются только при:

- `DROP DATABASE`
- Удалении файла БД (если SQLite)
- Ручном удалении записей

Как быстро проверить все компоненты?

bash

- *# Создайте файл `check.bat` с содержимым:*
- `@echo off`
- `echo` Проверка Python...
- `python --version`
- `echo` Проверка PostgreSQL...
- `psql -U postgres -c "\l" -q`
- `echo` Проверка виртуального окружения...
- `call venv\Scripts\activate && pip list`
- `echo` Проверка Django...
- `python manage.py check`

pause

Полный цикл развертывания за 5 минут

Скопируйте и выполните эти команды последовательно:

```
bash
```

- *# 1. Создайте папку для проекта*
- `mkdir C:\Projects`
- `cd C:\Projects`
-
- *# 2. Скопируйте проект (или клонируйте)*
- *# (вставьте сюда ваш способ копирования)*
-
- *# 3. Создайте виртуальное окружение*
- `cd kartotekaProject`
- `python -m venv venv`
- `venv\Scripts\activate`
-
- *# 4. Установите зависимости*
- `pip install django psycopg2-binary chardet networkx`
-
- *# 5. Создайте базу данных*
- `psql -U postgres -c "CREATE DATABASE kartoteka_db;"`
-
- *# 6. Примените миграции*
- `python manage.py migrate`
-
- *# 7. Создайте администратора*
- `python manage.py createsuperuser --username admin --email admin@example.com`
-
- *# 8. Запустите сервер*

```
python manage.py runserver
```

Важные замечания

Пароли везде одинаковые: 12345678

- PostgreSQL: postgres / 12345678
- Django admin: admin / 12345678
- База данных: kartoteka_db

Порты по умолчанию:

- PostgreSQL: 5432
- Django: 8000

Пути по умолчанию:

- Проект: C:\Users\%USERNAME%\PycharmProjects\kartotekaProject\
- Статика: core/static/core/images/guide/

Проверка работоспособности

После выполнения всех шагов откройте:

Страница	URL	Логин	Пароль
Главная	http://127.0.0.1:8000/	-	-
Админка	http://127.0.0.1:8000/admin/	admin	12345678
Руководство	http://127.0.0.1:8000/user-guide/	-	-

Если все работает — проект успешно развернут!

